VISVESVARAYA TECHNOLOGICAL UNIVERSITY
"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Artificial Intelligence (23CS5PCAIN)

Submitted by

Navaneeth V N (1BM22CS171)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Artificial Intelligence (23CS5PCAIN)" carried out by **Navaneeth VN (1BM22CS171),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Artificial Intelligence (23CS5PCAIN) work prescribed for the said degree.

Dr. Pallavi G B Associate Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

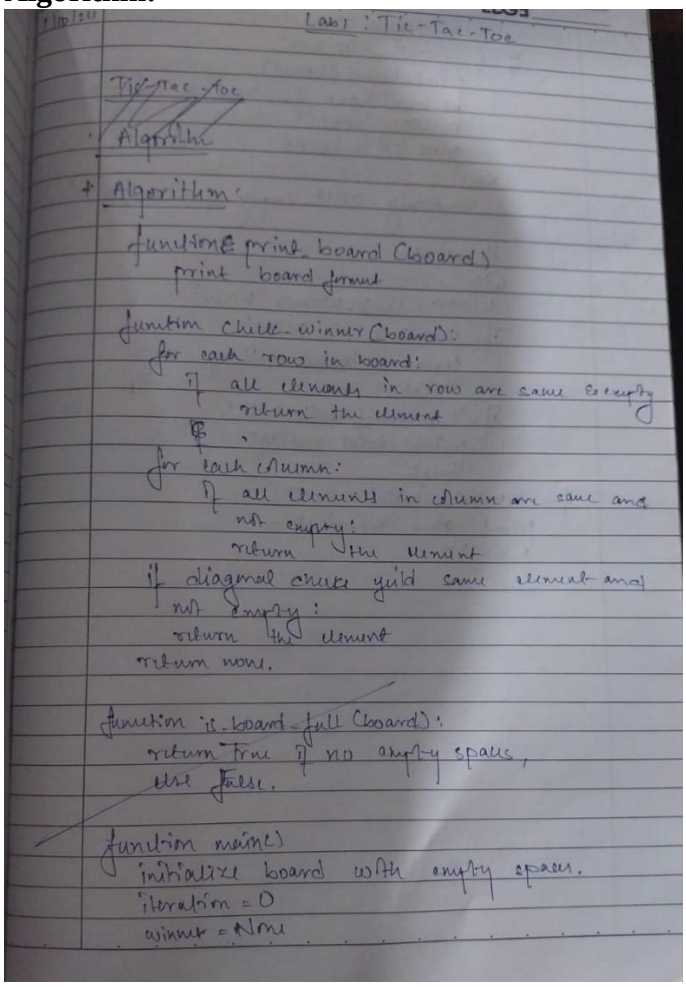
Sl. No.	Experiment Title	Page No.
1	Implement Tic –Tac –Toe Game Implement vacuum cleaner agent	4-9
2	Implement 8 puzzle problems using Depth First Search (DFS) Implement Iterative deepening search algorithm	10-15
3	Implement A* search algorithm	16-24
4	Implement Hill Climbing search algorithm to solve N-Queens problem	25-28
5	Simulated Annealing to Solve 8-Queens problem	29-31
6	Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.	32-33
7	Implement unification in first order logic	34-36
8	Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.	37-39
9	Create a knowledge base consisting of first order logic statements and prove the given query using Resolution	40-42
10	Implement Alpha-Beta Pruning.	43-45

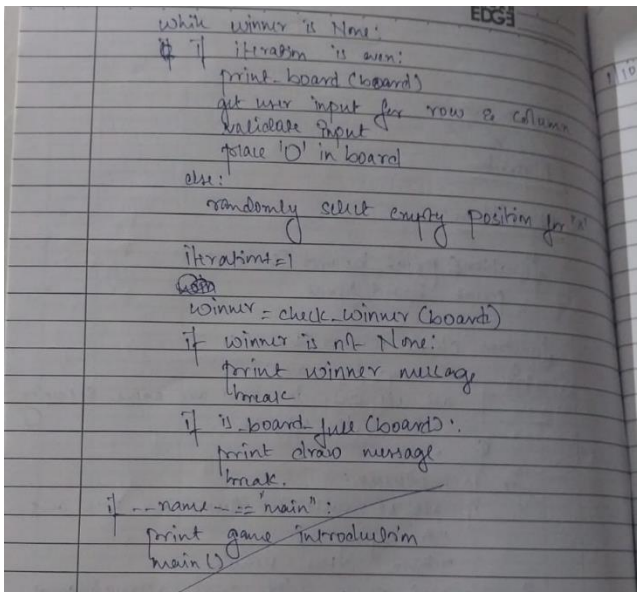
Github Link: https://github.com/bannanaplease/NavaneethVN_1BM22Cs171_AI_Lab

Program 1

Implement Tic –Tac –Toe Game

Algorithm:





Code:

```
import random
def check(row, col):
    if arr[row][col] == '_':
        return True
    else:
        return False

def check2():
    for i in range(3):
        for j in range(3):
            if arr[i][j] == '_':
                return True
    return False

def print_board():
    for row in arr:
        print(" | ".join(row))
        print("-" * 9)

def check_win(player):
    for i in range(3):
        if all([arr[i][j] == player for j in range(3)]):
            return True
        if all([arr[j][i] == player for j in range(3)]):
            return True
    if arr[0][0] == player and arr[1][1] == player and arr[2][2] == player:
        return True
    if arr[0][2] == player and arr[1][1] == player and arr[2][0] == player:
        return True
    return False

arr = [['_', '_', '_'],
       ['_', '_', '_'],
       ['_', '_', '_']]
print_board()
while check2():
    row = int(input("Enter row (0-2): "))
    col = int(input("Enter column (0-2): "))
    while not check(row, col):
        print("Place already occupied, try again.")
        row = int(input("Enter row (0-2): "))
        col = int(input("Enter column (0-2): "))
    arr[row][col] = 'X'
    print_board()
    if check_win('X'):
```

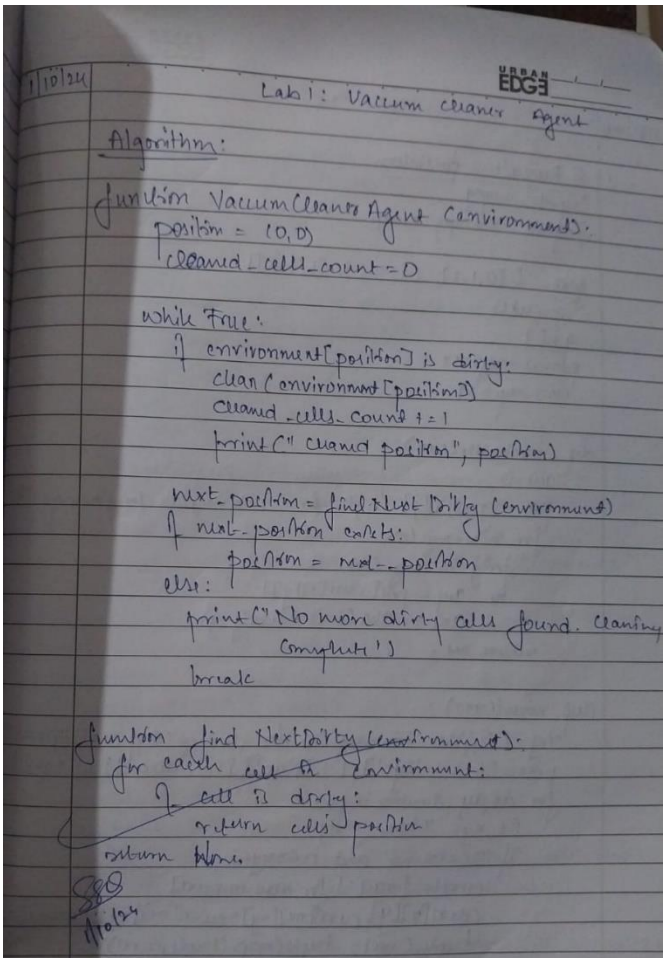
```
        print("Congratulations! You win!")
        break
    if not check2():
        print("It's a draw!")
        break
    print("Computer's turn...")
    row, col = random.randint(0, 2), random.randint(0, 2)
    while not check(row, col):
        row, col = random.randint(0, 2), random.randint(0, 2)
    arr[row][col] = 'O'
    print_board()
    if check_win('O'):
        print("Computer wins! Better luck next time.")
        break
    if not check2():
        print("It's a draw!")
        break
```

Output Snapshot:

```
_ | _ | _
---------
_ | _ | _
---------
_ | _ | _
---------
Enter row (0-2): 1
Enter column (0-2): 1
_ | _ | _
---------
_ | X | _
---------
_ | _ | _
---------
Computer's turn...
_ | O | _
---------
_ | X | _
---------
_ | _ | _
---------
Enter row (0-2): 2
Enter column (0-2): 2
_ | O | _
---------
_ | X | _
---------
_ | _ | X
---------
Computer's turn...
_ | O | _
---------
_ | X | _
---------
O | _ | X
---------
Enter row (0-2): 0
Enter column (0-2): 0
X | O | _
---------
_ | X | _
---------
O | _ | X
---------
Congratulations! You win!
```

Implement vacuum cleaner agent

Algorithm:



Code:

```
class VacuumCleaner:
    def __init__(self, room_a_dirt, room_b_dirt, starting_room):
        self.current_state = (room_a_dirt, room_b_dirt, starting_room)

    def is_goal_state(self):
        return self.current_state[0] == 0 and self.current_state[1] == 0

    def clean(self):
        if self.current_state[0] == 1:
            self.current_state = (0, self.current_state[1], self.current_state[2])
            print("Cleaned room A.")
        elif self.current_state[1] == 1:
            self.current_state = (self.current_state[0], 0, self.current_state[2])
```

```
            print("Cleaned room B.")

    def move(self):
        if self.current_state[2] == 'A':
            self.current_state = (self.current_state[0], self.current_state[1], 'B')
            print("Moved to room B.")
        else:
            self.current_state = (self.current_state[0], self.current_state[1], 'A')
            print("Moved to room A.")

    def run(self):
        while not self.is_goal_state():
            print(f"Current state: {self.current_state}")
            self.clean()
            if not self.is_goal_state():
                self.move()
        print("Both rooms are clean!")

def get_initial_state():
    room_a_dirt = int(input("Is room A dirty? (1 for yes, 0 for no): "))
    room_b_dirt = int(input("Is room B dirty? (1 for yes, 0 for no): "))
    starting_room = input("Which room is the vacuum cleaner in? (A or B): ").strip().upper()
    if starting_room not in ['A', 'B'] or room_a_dirt not in [0, 1] or room_b_dirt not in [0, 1]:
        print("Invalid input. Please enter the correct values.")
        return get_initial_state()

    return room_a_dirt, room_b_dirt, starting_room

initial_state = get_initial_state()
vacuum = VacuumCleaner(*initial_state)
vacuum.run()
```

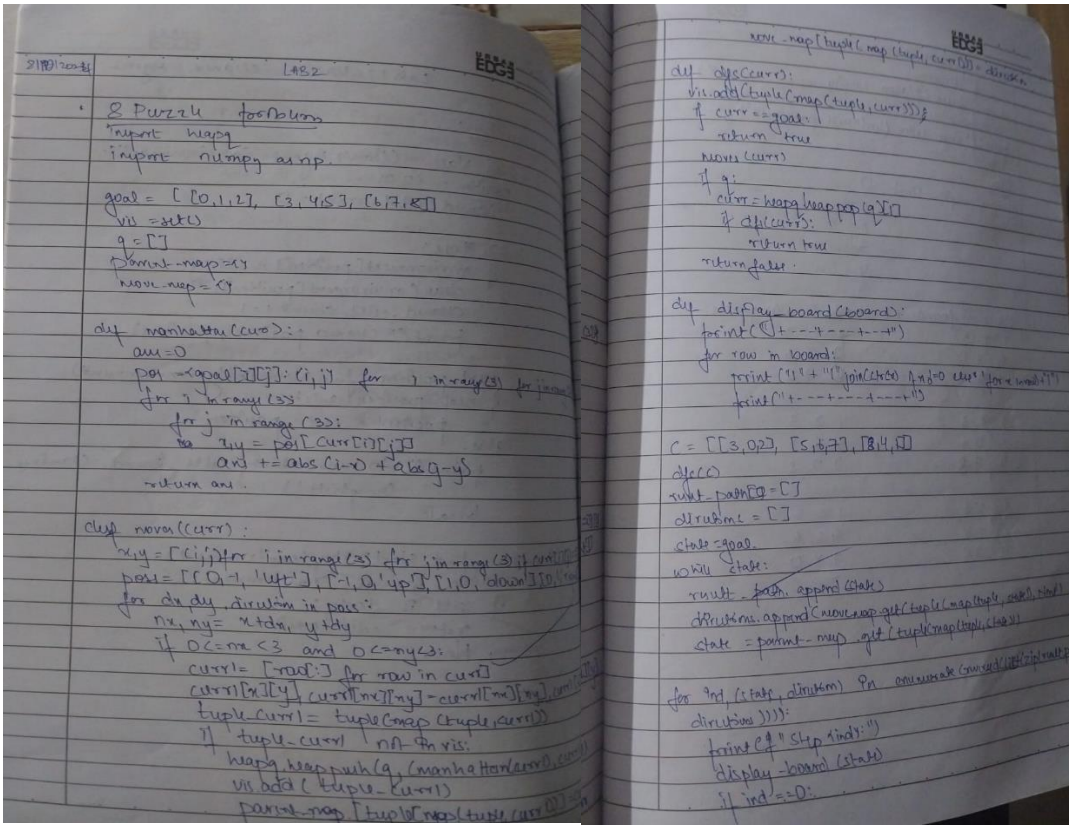
Output Snapshot:

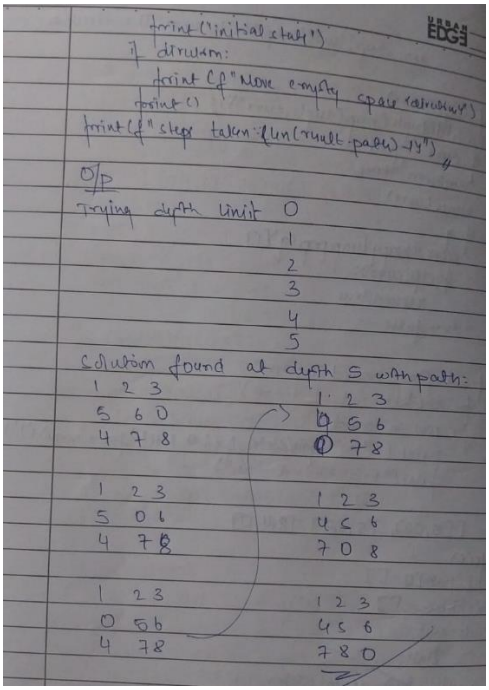
```
Is room A dirty? (1 for yes, 0 for no): 1
Is room B dirty? (1 for yes, 0 for no): 1
Which room is the vacuum cleaner in? (A or B): A
Current state: (1, 1, 'A')
Cleaned room A.
Moved to room B.
Current state: (0, 1, 'B')
Cleaned room B.
Both rooms are clean!
```

Program2:

Implement 8 puzzle problems using Depth First Search (DFS)

Algorithm:





Code:

```
from copy import deepcopy
goal_state = [[0, 1, 2],
              [3, 4, 5],
              [6, 7, 8]]
moves = {
    'up': (-1, 0),
    'down': (1, 0),
    'left': (0, -1),
    'right': (0, 1)
}

def find_blank(board):
    for i in range(3):
        for j in range(3):
            if board[i][j] == 0:
                return i, j
    return None

def is_goal(state):
    return state == goal_state

def is_valid_move(x, y):
    return 0 <= x < 3 and 0 <= y < 3

def apply_move(board, move):
    x, y = find_blank(board)
    dx, dy = moves[move]
    new_x, new_y = x + dx, y + dy
    if is_valid_move(new_x, new_y):
        new_board = deepcopy(board)

        new_board[x][y], new_board[new_x][new_y] = new_board[new_x][new_y],
new_board[x][y]
        return new_board
    return None

def dfs(start):
    stack = [(start, [])]
    visited = set()

    while stack:
        current_state, path = stack.pop()

        if is_goal(current_state):
            return path + [current_state]
```

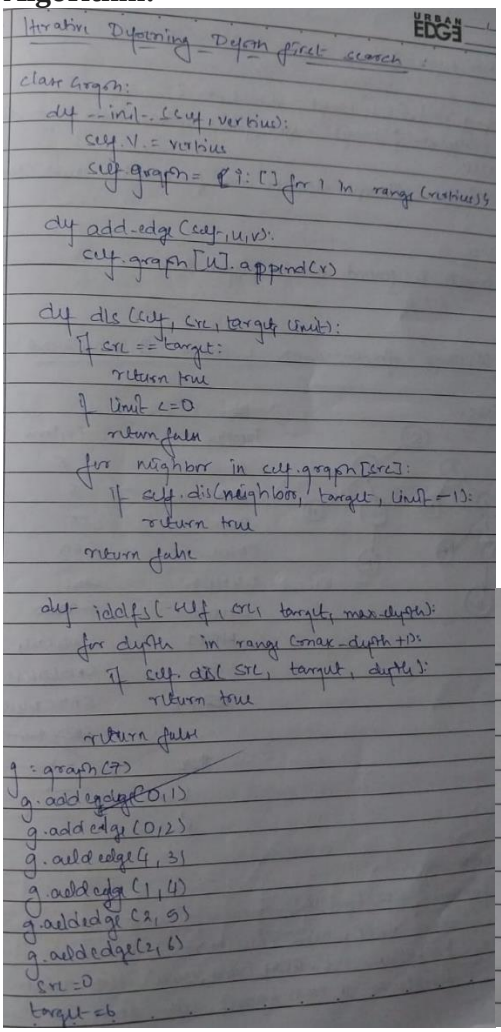
```
        visited.add(tuple(tuple(row) for row in current_state))

        for move in moves:
            new_state = apply_move(current_state, move)
            if new_state and tuple(tuple(row) for row in new_state) not in visited:
                stack.append((new_state, path + [current_state]))

    return None

def print_board(board):
    for row in board:
        print(row)
    print()

def print_solution(solution):
    if solution:
        for board in solution:
            print_board(board)
    else:
        print("No solution found")

initial_state = [[1, 2, 0],
                 [3, 4, 5],
                 [6, 7, 8]]

solution = dfs(initial_state)

print_solution(solution)
```

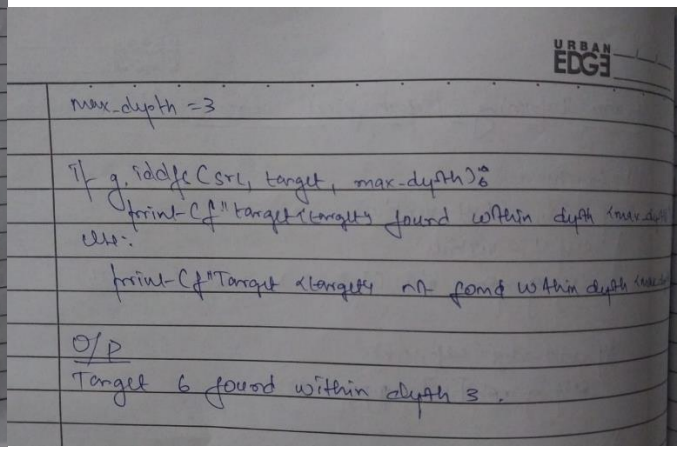
Output Snapshot:

```
[1, 2, 0]
[3, 4, 5]
[6, 7, 8]

[1, 0, 2]
[3, 4, 5]
[6, 7, 8]

[0, 1, 2]
[3, 4, 5]
[6, 7, 8]
```

Implement Iterative deepening search algorithm

Algorithm:





Code:

```
from copy import deepcopy
goal_state = [[0, 1, 2],
              [3, 4, 5],
              [6, 7, 8]]
moves = {
    'up': (-1, 0),
    'down': (1, 0),
    'left': (0, -1),
    'right': (0, 1)
}

def find_blank(board):
    for i in range(3):
        for j in range(3):
```

```
            if board[i][j] == 0:
                return i, j
    return None

def is_goal(state):
    return state == goal_state

def is_valid_move(x, y):
    return 0 <= x < 3 and 0 <= y < 3

def apply_move(board, move):
    x, y = find_blank(board)
    dx, dy = moves[move]
    new_x, new_y = x + dx, y + dy
    if is_valid_move(new_x, new_y):
        new_board = deepcopy(board)
        new_board[x][y], new_board[new_x][new_y] = new_board[new_x][new_y],
new_board[x][y]
        return new_board
    return None

def dfs_limited(state, path, depth_limit, visited):
    if is_goal(state):
        return path + [state]

    if depth_limit == 0:
        return None

    visited.add(tuple(tuple(row) for row in state))

    for move in moves:
        new_state = apply_move(state, move)
        if new_state and tuple(tuple(row) for row in new_state) not in visited:
            result = dfs_limited(new_state, path + [state], depth_limit - 1, visited)
            if result:
                return result

    visited.remove(tuple(tuple(row) for row in state))
    return None

def ids(start):
    depth_limit = 0
    while True:
        visited = set()
        result = dfs_limited(start, [], depth_limit, visited)
        if result:
            return result
```

```
        depth_limit += 1

def print_board(board):
    for row in board:
        print(row)
    print()

def print_solution(solution):
    if solution:
        for board in solution:
            print_board(board)
    else:
        print("No solution found")

initial_state = [[1, 2, 5],
                 [0, 3, 8],
                 [6, 4, 7]]
solution = ids(initial_state)
print_solution(solution)
```

Output Snapshot:

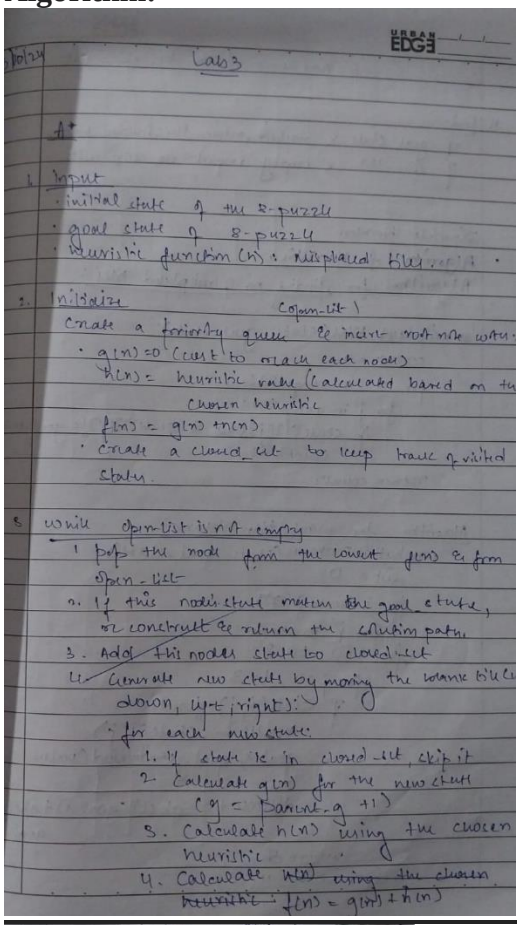
```
[1, 2, 5]
[0, 3, 8]
[6, 4, 7]

[1, 2, 5]
[3, 0, 8]
[6, 4, 7]

[1, 2, 5]
[3, 4, 8]
[6, 0, 7]

[1, 2, 5]
[3, 4, 8]
[6, 7, 0]

[1, 2, 5]
[3, 4, 0]
[6, 7, 8]

[1, 2, 0]
[3, 4, 5]
[6, 7, 8]

[1, 0, 2]
[3, 4, 5]
[6, 7, 8]

[0, 1, 2]
[3, 4, 5]
[6, 7, 8]
```

Program3:

Implement A* search algorithm

i) Misplaced tiles

Algorithm:

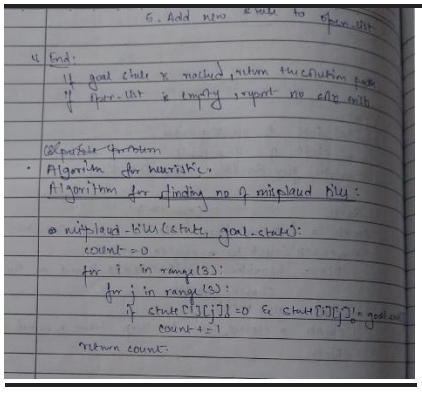

Lab 3

A*

1. Input
- initial state of the 8-puzzle
- goal state of 8-puzzle
- Heuristic function (h): misplaced tiles

2. Initialize
Create a priority queue (open_list)
- g(n) = 0 (cost to reach each node)
- h(n) = heuristic value (calculated based on the chosen heuristic)
- f(n) = g(n) + h(n)
- Create a closed_list to keep track of visited states

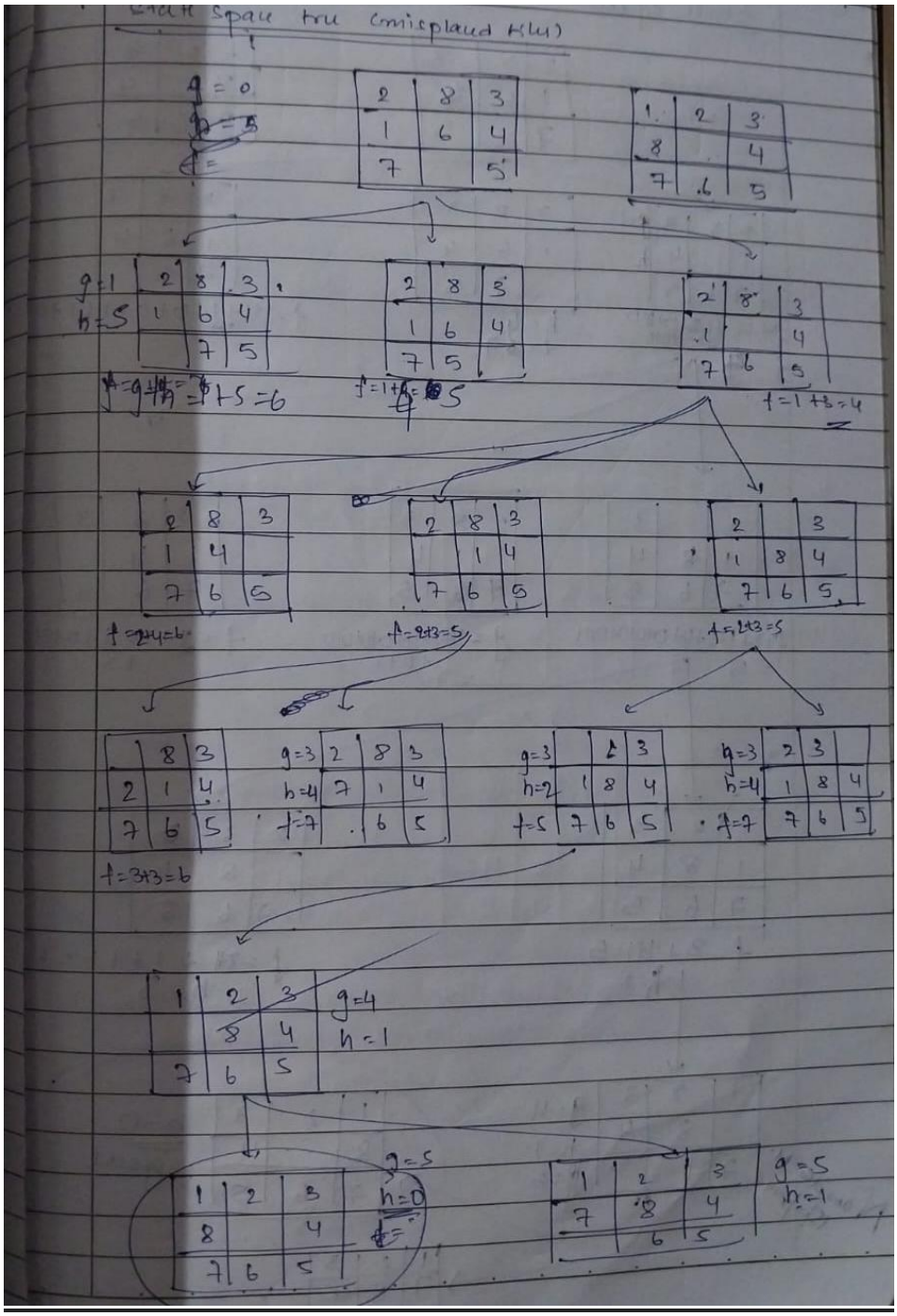
3. while open_list is not empty
1. Pop the node from the lowest f(n) from open_list
2. If this node's state matches the goal_state, reconstruct & return the solution path.
3. Add this node's state to closed_list
4. Generate new states by moving the blank tile (up, down, left, right):
- for each new state:
1. If state is in closed_list, skip it
2. Calculate g(n) for the new state (g = parent_g + 1)
3. Calculate h(n) using the chosen heuristic
4. Calculate f(n) using the chosen heuristic: f(n) = g(n) + h(n)


5. Add new state to open_list

4. End:
If goal state is reached, return the solution path
If open_list is empty, report no solution

Algorithm for heuristic:
Algorithm for finding no of misplaced tiles:

```
misplaced_tiles(state, goal_state):
    count = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 & state[i][j] != goal_state[i][j]:
                count += 1
    return count
```



Code:

```
class Node:
    def __init__(self, data, level, fval):
        self.data = data
        self.level = level
        self.fval = fval

    def generate_child(self):
```

```
        x, y = self.find_blank()
        moves = [(x, y-1), (x, y+1), (x-1, y), (x+1, y)]
        children = []
        for new_x, new_y in moves:
            child_data = self.move_blank(x, y, new_x, new_y)
            if child_data:
                child_node = Node(child_data, self.level + 1, 0)
                children.append(child_node)
        return children

    def move_blank(self, x1, y1, x2, y2):
        if 0 <= x2 < len(self.data) and 0 <= y2 < len(self.data[0]):
            new_data = [row[:] for row in self.data]
            new_data[x1][y1], new_data[x2][y2] = new_data[x2][y2], new_data[x1][y1]
            return new_data
        return None

    def find_blank(self):
        for i, row in enumerate(self.data):
            if '_' in row:
                return i, row.index('_')


class Puzzle:
    def __init__(self, size):
        self.size = size
        self.open = []
        self.closed = []

    def get_input(self):
        return [input().split() for _ in range(self.size)]

    def f(self, start, goal):
        return start.level + self.h(start.data, goal)

    def h(self, start_data, goal):
        return sum(start_data[i][j] != goal[i][j] and start_data[i][j] != '_' for i in
range(self.size) for j in range(self.size))

    def process(self):
        print("Enter the start state matrix:")
        start_data = self.get_input()
        print("Enter the goal state matrix:")
        goal = self.get_input()

        start_node = Node(start_data, 0, 0)
        start_node.fval = self.f(start_node, goal)
```

```
        self.open.append(start_node)

        while self.open:
            current_node = self.open.pop(0)
            self.display_state(current_node, goal)

            if self.h(current_node.data, goal) == 0:
                print("Goal reached!")
                break

            children = current_node.generate_child()
            for child in children:
                child.fval = self.f(child, goal)
                self.open.append(child)

            self.closed.append(current_node)
            self.open.sort(key=lambda node: node.fval)

    def display_state(self, node, goal):
        print("\nNext step:")
        for row in node.data:
            print(" ".join(row))
        heuristic_value = self.h(node.data, goal)
        print(f"Heuristic (h): {heuristic_value}")
        print(f"Depth (g): {node.level}")
        print(f"Function value (f = g + h): {node.fval}")
puz = Puzzle(3)
puz.process()
```

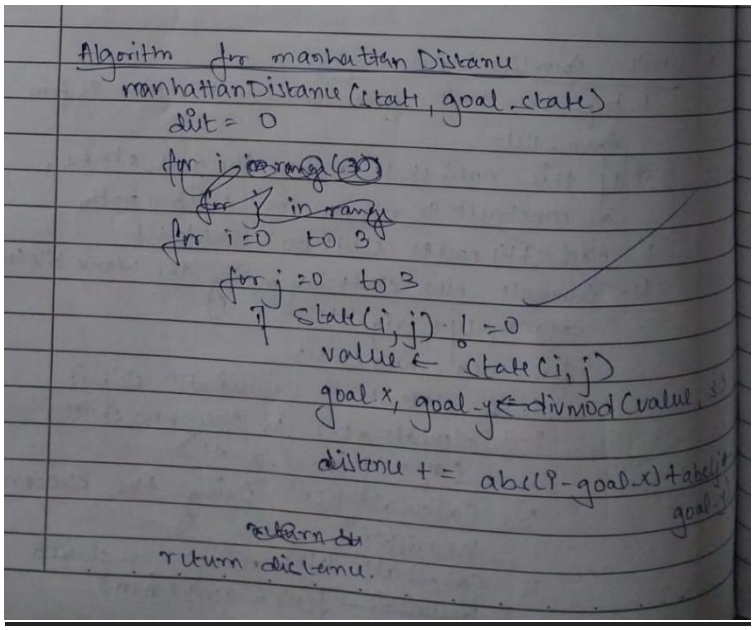
Output Snapshot:

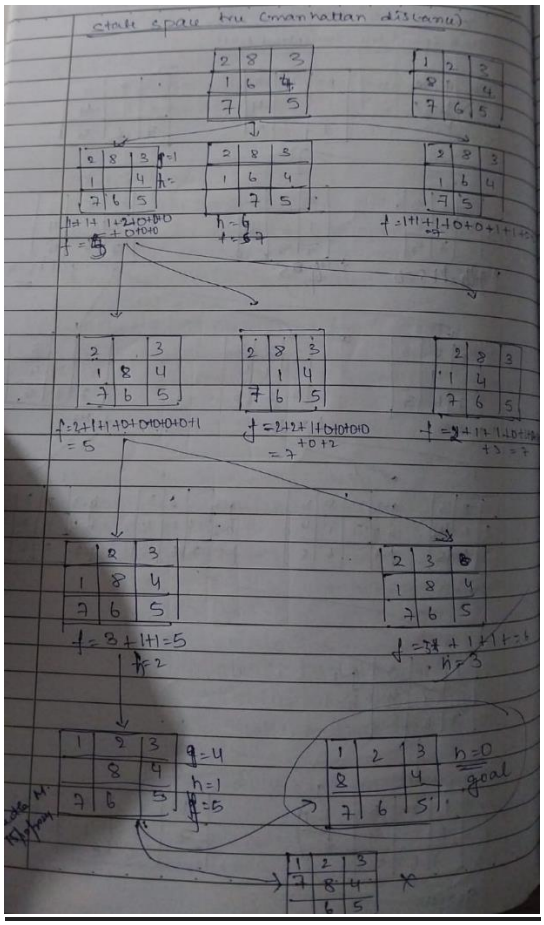
```
Enter the start state matrix:
2 8 3
1 6 4
7 _ 5
Enter the goal state matrix:
1 2 3
8 _ 4
7 6 5

Next step:
2 8 3
1 6 4
7 _ 5
Heuristic (h): 4
Depth (g): 0
Function value (f = g + h): 4

Next step:
2 8 3
1 _ 4
7 6 5
Heuristic (h): 3
Depth (g): 1
Function value (f = g + h): 4

Next step:
2 8 3
_ 1 4
7 6 5
Heuristic (h): 3
Depth (g): 2
Function value (f = g + h): 5

Next step:
2 _ 3
1 8 4
7 6 5
Heuristic (h): 3
Depth (g): 2
Function value (f = g + h): 5
```

```
Next step:
_ 2 3
1 8 4
7 6 5
Heuristic (h): 2
Depth (g): 3
Function value (f = g + h): 5

Next step:
1 2 3
_ 8 4
7 6 5
Heuristic (h): 1
Depth (g): 4
Function value (f = g + h): 5

Next step:
1 2 3
8 _ 4
7 6 5
Heuristic (h): 0
Depth (g): 5
Function value (f = g + h): 5
Goal reached!
```

ii) Manhattan distance

Algorithm:





Code:

```
class Node:
    def __init__(self, data, level, fval):
        self.data = data
        self.level = level
        self.fval = fval

    def generate_child(self):
        x, y = self.find_blank()
        moves = [(x, y-1), (x, y+1), (x-1, y), (x+1, y)]
        children = []
        for new_x, new_y in moves:
            child_data = self.move_blank(x, y, new_x, new_y)
            if child_data:
                child_node = Node(child_data, self.level + 1, 0)
                children.append(child_node)
        return children

    def move_blank(self, x1, y1, x2, y2):
        if 0 <= x2 < len(self.data) and 0 <= y2 < len(self.data[0]):
            new_data = [row[:] for row in self.data]
            new_data[x1][y1], new_data[x2][y2] = new_data[x2][y2], new_data[x1][y1]
            return new_data
        return None

    def find_blank(self):
        for i, row in enumerate(self.data):
            if '_' in row:
                return i, row.index('_')


class Puzzle:
    def __init__(self, size):
        self.size = size
        self.open = []
        self.closed = []

    def get_input(self):
        return [input().split() for _ in range(self.size)]

    def f(self, start, goal):
        h_val = self.manhattan_heuristic(start.data, goal)
        return start.level + h_val, h_val

    def manhattan_heuristic(self, start_data, goal):
        distance = 0
        for i in range(self.size):
```

```
            for j in range(self.size):
                if start_data[i][j] != '_' and start_data[i][j] != goal[i][j]:
                    goal_x, goal_y = self.find_position(goal, start_data[i][j])
                    distance += abs(i - goal_x) + abs(j - goal_y)
        return distance

    def find_position(self, state, value):
        for i in range(self.size):
            for j in range(self.size):
                if state[i][j] == value:
                    return i, j

    def process(self):
        print("Enter the start state matrix:")
        start_data = self.get_input()
        print("Enter the goal state matrix:")
        goal = self.get_input()

        start_node = Node(start_data, 0, 0)
        start_node.fval, h_val = self.f(start_node, goal)
        self.open.append(start_node)

        while self.open:
            current_node = self.open.pop(0)
            self.display_state(current_node.data, current_node.fval, h_val, current_node.level)

            if self.manhattan_heuristic(current_node.data, goal) == 0:
                print("Goal reached!")
                break

            children = current_node.generate_child()
            for child in children:
                child.fval, h_val = self.f(child, goal)
                self.open.append(child)

            self.closed.append(current_node)
            self.open.sort(key=lambda node: node.fval)

    def display_state(self, data, f_val, h_val, g_val):
        print("\nNext step:")
        for row in data:
            print(" ".join(row))
        print(f"f(x) = {f_val} (g(x) = {g_val}, h(x) = {h_val})")
puz = Puzzle(3)
puz.process()
```

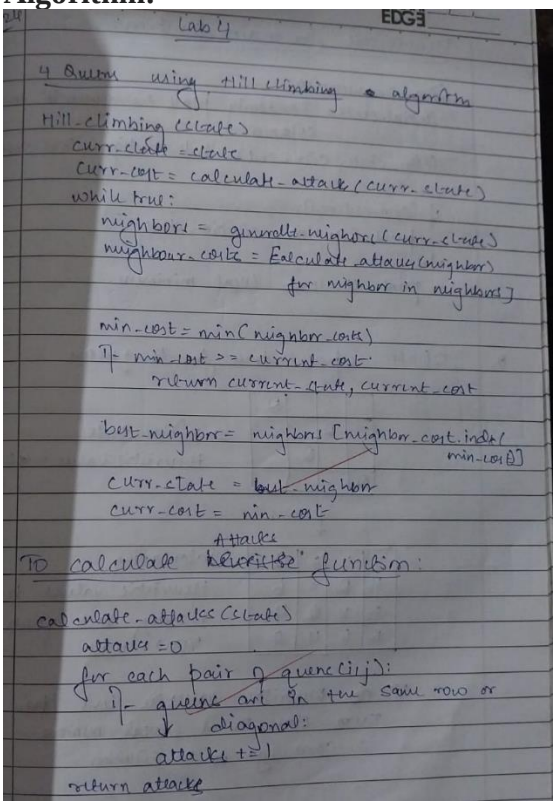
output snapshot:

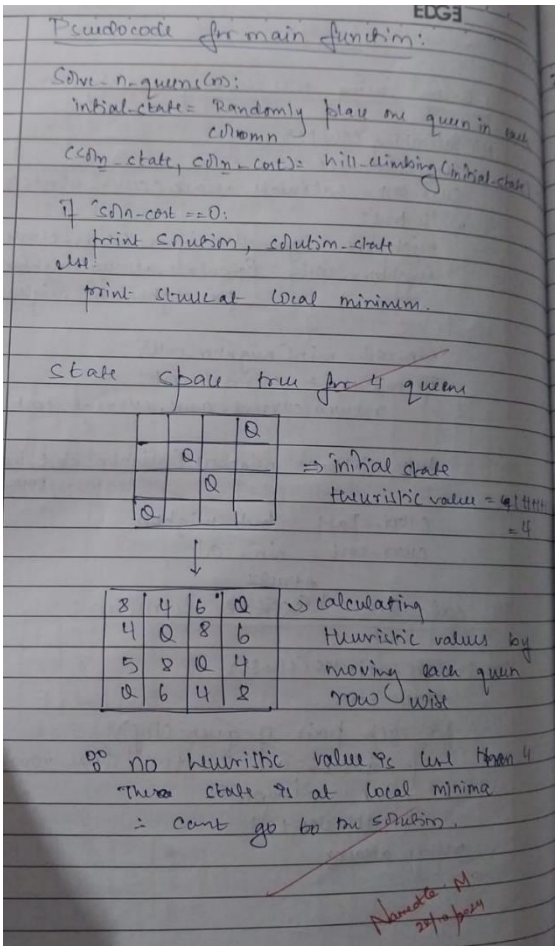
```
Enter the start state matrix:
2 8 3
1 6 4
_ 7 5
Enter the goal state matrix:
1 2 3
8 _ 4
7 6 5

Next step:
2 8 3
1 6 4
_ 7 5
f(x) = 6 (g(x) = 0, h(x) = 6)

Next step:
2 8 3
1 6 4
7 _ 5
f(x) = 6 (g(x) = 1, h(x) = 7)

Next step:
2 8 3
1 _ 4
7 6 5
f(x) = 6 (g(x) = 2, h(x) = 4)

Next step:
2 _ 3
1 8 4
7 6 5
f(x) = 6 (g(x) = 3, h(x) = 5)

Next step:
_ 2 3
1 8 4
7 6 5
f(x) = 6 (g(x) = 4, h(x) = 4)

Next step:
1 2 3
_ 8 4
7 6 5
f(x) = 6 (g(x) = 5, h(x) = 1)

Next step:
1 2 3
8 _ 4
7 6 5
f(x) = 6 (g(x) = 6, h(x) = 2)
Goal reached!
```

Program4:

Implement Hill Climbing search algorithm to solve N-Queens problem

Algorithm:





Code:

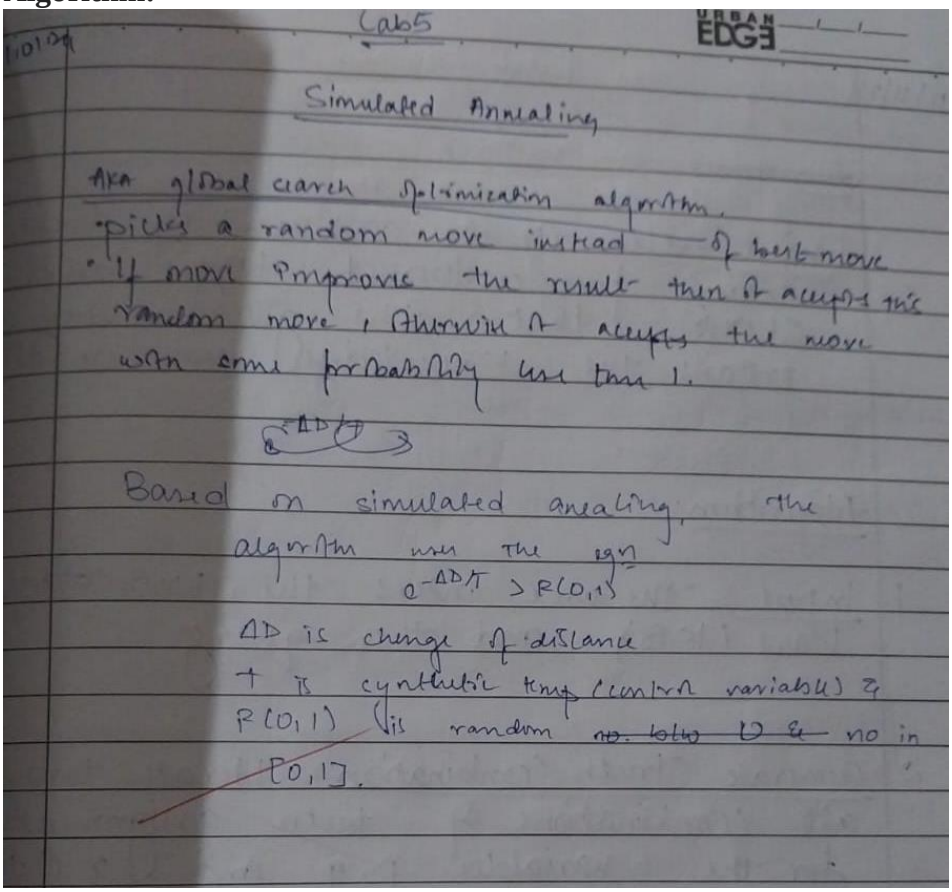
```
def calculateCost(state):
    attacking_pairs = 0
    for i in range(N):
        for j in range(i + 1, N):
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                attacking_pairs += 1
    return attacking_pairs

def getNeighbours(state):
    neighbours = []
    for i in range(N):
        for j in range(i + 1, N):
            new_state = state[:]
            new_state[i], new_state[j] = new_state[j], new_state[i]
            neighbours.append(new_state)
    return neighbours

def hillClimbing(initial_state):
    current_state = initial_state
    current_cost = calculateCost(current_state)
```

```
    iteration = 0
    while True:
        print(f"\nIteration {iteration}")
        print(f"Current State: {current_state}, Cost: {current_cost}")

        neighbours = getNeighbours(current_state)
        next_state = current_state
        next_cost = current_cost

        for neighbour in neighbours:
            cost = calculateCost(neighbour)
            print(f"Neighbour: {neighbour}, Cost: {cost}")
            if cost < next_cost:
                next_state = neighbour
                next_cost = cost

        if next_cost == current_cost:
            break
        else:
            current_state, current_cost = next_state, next_cost

        if current_cost == 0:
            break

        iteration += 1

    return current_state, current_cost

N=4 #Board Size
initial_state = list(map(int, input("Enter initial state as space-separated integers (0-based
indexing): ").split()))
solution_state, solution_cost = hillClimbing(initial_state)

print("\nFinal Results")
print("Initial State:", initial_state)
print("Final State (Solution):", solution_state)
print("Final Cost (Attacking Pairs):", solution_cost)

if solution_cost == 0:
    print("Solution found!")
else:
    print("Local optimum reached, but no solution.")
```
Output Snapshot:

```
Enter initial state as space-separated integers (0-based indexing): 3 1 2 0

Iteration 0
Current State: [3, 1, 2, 0], Cost: 2
Neighbour: [1, 3, 2, 0], Cost: 1
Neighbour: [2, 1, 3, 0], Cost: 1
Neighbour: [0, 1, 2, 3], Cost: 6
Neighbour: [3, 2, 1, 0], Cost: 6
Neighbour: [3, 0, 2, 1], Cost: 1
Neighbour: [3, 1, 0, 2], Cost: 1

Iteration 1
Current State: [1, 3, 2, 0], Cost: 1
Neighbour: [3, 1, 2, 0], Cost: 2
Neighbour: [2, 3, 1, 0], Cost: 2
Neighbour: [0, 3, 2, 1], Cost: 4
Neighbour: [1, 2, 3, 0], Cost: 4
Neighbour: [1, 0, 2, 3], Cost: 2
Neighbour: [1, 3, 0, 2], Cost: 0

Final Results
Initial State: [3, 1, 2, 0]
Final State (Solution): [1, 3, 0, 2]
Final Cost (Attacking Pairs): 0
Solution found!
```

Program5:
Simulated Annealing to Solve 8-Queens problem

Algorithm:



Code:

```
import random
import math

n=8

def no_of_conflicts(board):
    conflicts = 0
    n = len(board)
    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j] or abs(board[i] - board[j]) == j - i:
                conflicts += 1
    return conflicts

def initialize(n):
    return [random.randint(0, n - 1) for _ in range(n)]
```

```
def simulated_annealing(n, max_iterations=10000, initial_temp=100, cooling_rate=0.99):
    board = initialize(n)
    current_conflicts = no_of_conflicts(board)
    temperature = initial_temp

    for iteration in range(max_iterations):
        if current_conflicts == 0:
            print(f"Iteration {iteration}: Solution found!")
            print("Board Position:", board)
            return board

        row = random.randint(0, n - 1)
        new_col = random.randint(0, n - 1)
        while new_col == board[row]:
            new_col = random.randint(0, n - 1)

        new_board = board[:]
        new_board[row] = new_col
        new_conflicts = no_of_conflicts(new_board)
        delta_conflicts = new_conflicts - current_conflicts
        if delta_conflicts < 0 or math.exp(-delta_conflicts / temperature) > random.random():
            board, current_conflicts = new_board, new_conflicts
        print(f"Iteration {iteration}: Temperature={temperature:.2f}, Current
Conflicts={current_conflicts}, "
              f"Delta Conflicts={delta_conflicts}, New Position for Row {row} -> Column
{new_col}")
        print("Board Position:", board)
        temperature *= cooling_rate

    print("No solution found within the maximum iterations.")
    return None

solution = simulated_annealing(n)
if solution:
    print("Final Solution:", solution)
else:
    print("No solution found within the maximum iterations.")
```

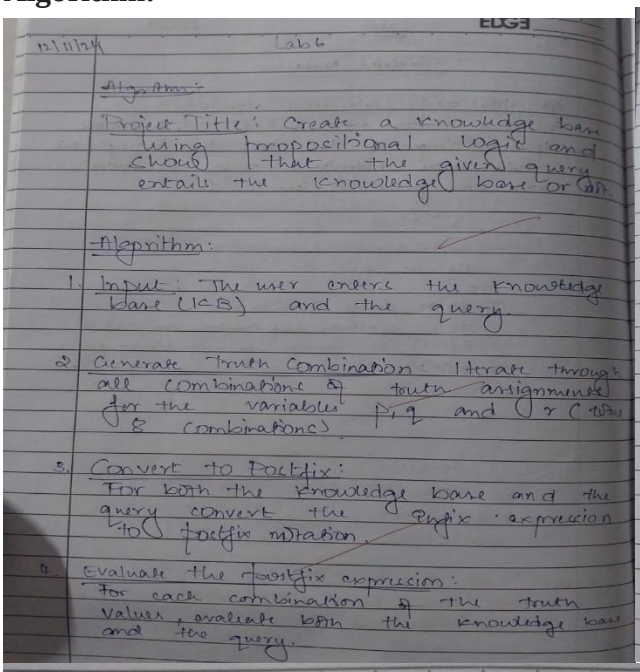
Output Snapshot:

```
Board Position: [4, 6, 3, 5, 7, 1, 4, 2]
Iteration 1227: Temperature=0.00, Current Conflicts=1, Delta Conflicts=3, New Position for Row 4 -> Column 5
Board Position: [4, 6, 3, 5, 7, 1, 4, 2]
Iteration 1228: Temperature=0.00, Current Conflicts=1, Delta Conflicts=1, New Position for Row 0 -> Column 2
Board Position: [4, 6, 3, 5, 7, 1, 4, 2]
Iteration 1229: Temperature=0.00, Current Conflicts=1, Delta Conflicts=3, New Position for Row 7 -> Column 4
Board Position: [4, 6, 3, 5, 7, 1, 4, 2]
Iteration 1230: Temperature=0.00, Current Conflicts=1, Delta Conflicts=1, New Position for Row 0 -> Column 1
Board Position: [4, 6, 3, 5, 7, 1, 4, 2]
Iteration 1231: Temperature=0.00, Current Conflicts=0, Delta Conflicts=-1, New Position for Row 0 -> Column 0
Board Position: [0, 6, 3, 5, 7, 1, 4, 2]
Iteration 1232: Solution found!
Board Position: [0, 6, 3, 5, 7, 1, 4, 2]
Final Solution: [0, 6, 3, 5, 7, 1, 4, 2]
```

Program6:

Create a knowledge base using propositional logic and show that the given query entails the knowledge base or not.

Algorithm:



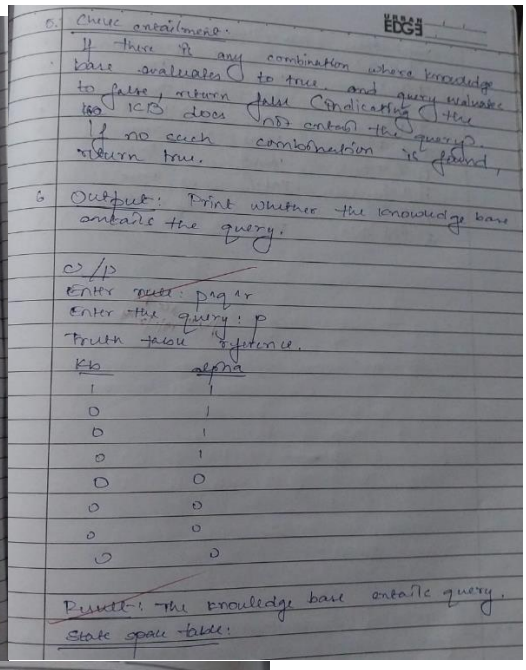
Lab 6

Algorithm:

Project Title: Create a knowledge base using propositional logic and check that the given query entails the knowledge base or not

Algorithm:

1. Input: The user enters the knowledge base (KB) and the query.
2. Generate Truth Combination: Iterate through all combinations of truth assignments for the variables p, q and r (8 combinations)
3. Convert to Postfix: For both the knowledge base and the query convert the prefix expression to postfix notation.
4. Evaluate the postfix expression: For each combination of the truth values, evaluate both the knowledge base and the query.

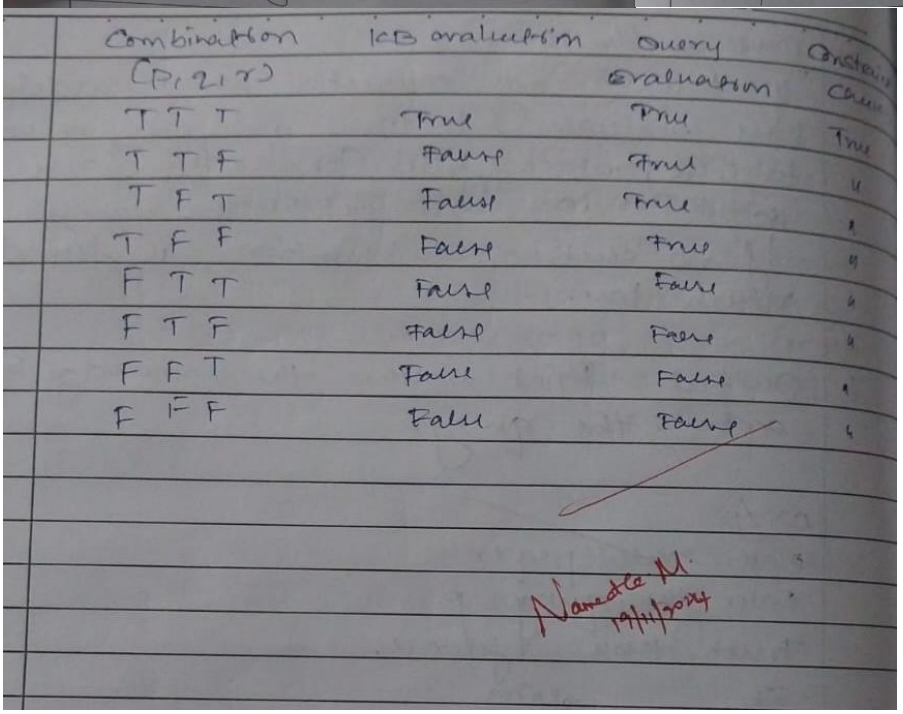


5. Check entailment: If there is any combination where knowledge base evaluates to true, and query evaluates to false, return false (Indicating the KB does not entail the query). If no such combination is found, return true.
6. Output: Print whether the knowledge base entails the query.

C:/>
Enter the query: p
Enter the query: p
Truth table reference:

KB	alpha
1	1
0	1
0	1
0	1
0	0
0	0
0	0
0	0

Result: The knowledge base entails query.
State space table:



Combination (P, Q, R)	KB evaluation	Query Evaluation	Conclusion
T T T	True	True	True
T T F	False	True	
T F T	False	True	
T F F	False	True	
F T T	False	False	
F T F	False	False	
F F T	False	False	
F F F	False	False	

Nandeesha M
19/11/2024

Code:

```
def implies(p, q):
    return not p or q
```

```
def print_truth_table(KB, alpha, sym):
    print("Truth Table:")
    header = " | ".join(sym) + " | KB | alpha "
    print(header)
    print("-" * len(header))

    for values in product([True, False], repeat=len(sym)):
        model = dict(zip(sym, values))
        kb_true = all(statement(model) for statement in KB)
        query_true = query(model)

        row = " | ".join(str(val) for val in values) + f" | {kb_true} | {query_true}"
        print(row)

def check_entailment(KB, query, sym):

    for values in product([True, False], repeat=len(sym)):
        model = dict(zip(sym, values))
        kb_true = all(statement(model) for statement in KB)
        query_true = query(model)
        if kb_true and not query_true:
            return False
    return True
sym=['P', 'Q', 'R']
KB = [
    lambda model: implies(model['Q'], model['P']),
    lambda model: implies(model['P'], not model['Q']),
    lambda model: model['Q'] or model['R']
]
alpha = lambda model: model['R']


entails = check_entailment(KB, alpha,sym)
print(f"KB entails alpha: {entails}")
print_truth_table(KB, alpha,sym)
```

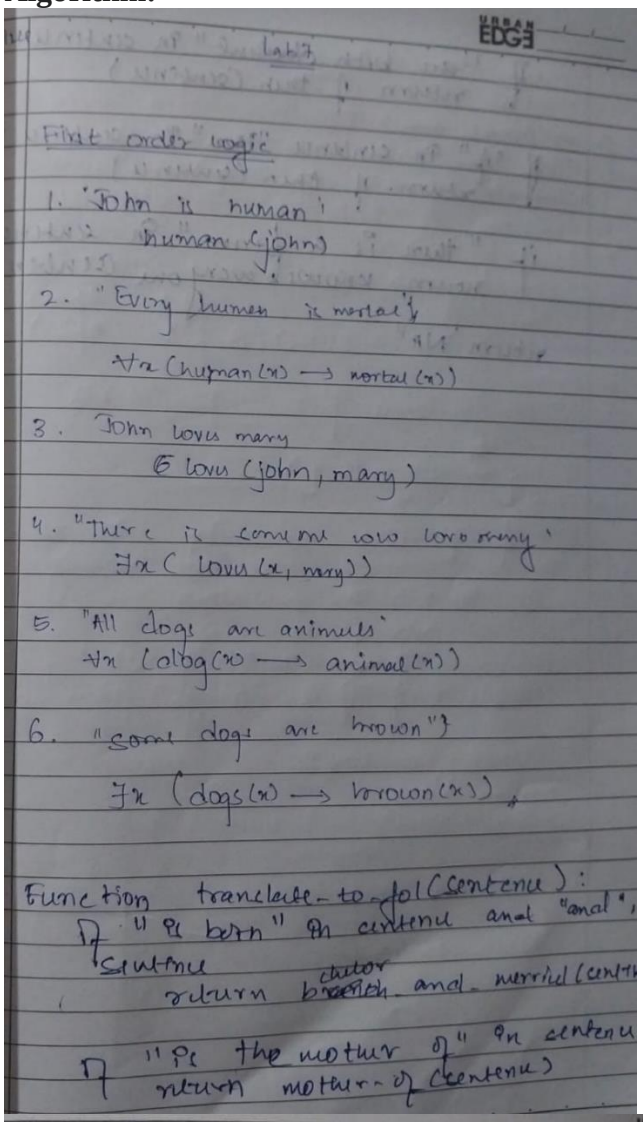
Output Snapshot:

```
KB entails alpha: True
Truth Table:
P | Q | R | KB | alpha
----------------------
True | True | True | False | True
True | True | False | False | False
True | False | True | True | True
True | False | False | False | False
False | True | True | False | True
False | True | False | False | False
False | False | True | True | True
False | False | False | False | False
```

sProgram7:

Implement unification in first order logic

Algorithm:

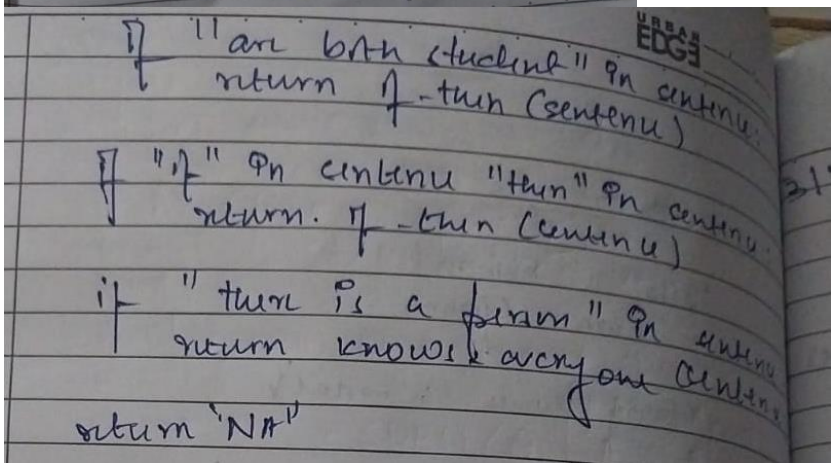


Lab 7

First order logic

1. "John is human"
   human(john)
2. "Every human is mortal"
   ∀x (human(x) → mortal(x))
3. John loves mary
   loves(john, mary)
4. "There is someone who loves mary"
   ∃x (loves(x, mary))
5. "All dogs are animals"
   ∀x (dog(x) → animal(x))
6. "some dogs are brown"
   ∃x (dogs(x) → brown(x))

function translate_to_fol(sentence):
if "is born" in sentence and "anal"
sentence
return brother and married(sentence)
if "is the mother of" in sentence
return mother_of(sentence)



if "are both students" in sentence:
return if_then(sentence)
if "if" in sentence "then" in sentence
return if_then(sentence)
if "then is a person" in sentence
return knows(x, everyone else)
return "NA"

Code:

```
def unify_terms(term_a, term_b, subs=None):
    if subs is None:
        subs = {}

    if term_a == term_b:
        return subs

    if is_variable(term_a):
        return unify_with_var(term_a, term_b, subs)
    if is_variable(term_b):
        return unify_with_var(term_b, term_a, subs)

    if is_compound(term_a) and is_compound(term_b):
        if term_a[0] != term_b[0] or len(term_a[1]) != len(term_b[1]):
            return None
        for subterm_a, subterm_b in zip(term_a[1], term_b[1]):
            subs = unify_terms(subterm_a, subterm_b, subs)
            if subs is None:
                return None
        return subs

    if isinstance(term_a, list) and isinstance(term_b, list):
        if len(term_a) != len(term_b):
            return None
        for element_a, element_b in zip(term_a, term_b):
            subs = unify_terms(element_a, element_b, subs)
            if subs is None:
                return None
        return subs

    return None
def unify_with_var(var, expr, subs):

    if var in subs:
        return unify_terms(subs[var], expr, subs)
    if expr in subs:
        return unify_terms(var, subs[expr], subs)
    if occurs_check(var, expr, subs):
        return None  # Cyclic substitution check failed
    subs[var] = expr
    return subs


def occurs_check(var, expr, subs):

    if var == expr:
```

```
        return True
    if is_compound(expr):
        return any(occurs_check(var, arg, subs) for arg in expr[1])
    if isinstance(expr, list):
        return any(occurs_check(var, item, subs) for item in expr)
    if expr in subs:
        return occurs_check(var, subs[expr], subs)
    return False


def is_variable(item):

    return isinstance(item, str) and item.startswith('?')


def is_compound(item):

    return isinstance(item, tuple) and len(item) == 2 and isinstance(item[1], list)


if __name__ == "__main__":
    print("Enter expressions in the following format:")
    print("Compound terms: ('f', ['a', 'b'])")
    print("Variables: '?x', '?y'")
    print("Lists: ['a', 'b']")
    print("Constants: 'a', 'b', etc.\n")

    term_1 = eval(input("Enter the first expression (Ψ₁): "))
    term_2 = eval(input("Enter the second expression (Ψ₂): "))

    substitution_result = unify_terms(term_1, term_2)
    if substitution_result is None:
        print("Unification failed!")
    else:
        print("Unification successful!")
        print("Substitution Set:", substitution_result)
```

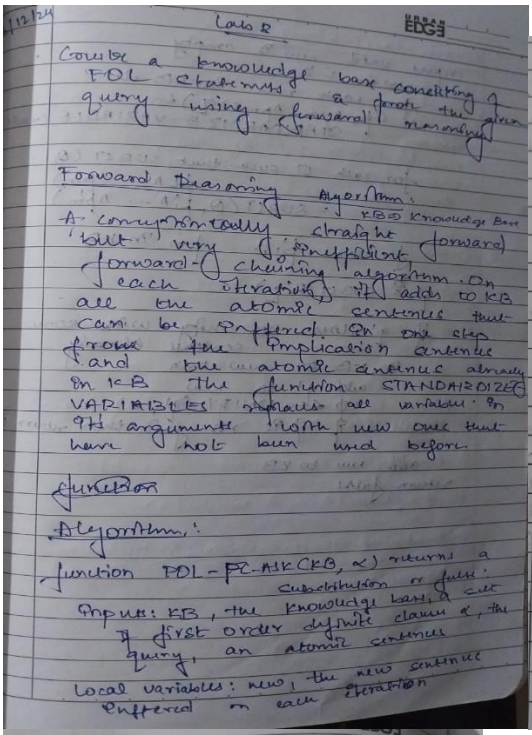
Output Snapshot:

```
Enter expressions in the following format:
Compound terms: ('f', ['a', 'b'])
Variables: '?x', '?y'
Lists: ['a', 'b']
Constants: 'a', 'b', etc.

Enter the first expression (Ψ₁): ('Studies',['Abubakar','?x'])
Enter the second expression (Ψ₂): ('Studies',['?y','AI'])
Unification successful!
Substitution Set: {'?y': 'Abubakar', '?x': 'AI'}
```

Program8:

Create a knowledge base consisting of first order logic statements and prove the given query using forward reasoning.

Algorithm:


Lab 8

Create a knowledge base consisting of FOL statements & prove the given query using forward reasoning

Forward Reasoning Algorithm:

A conceptually straight forward, but very inefficient forward-chaining algorithm. On each iteration, it adds to KB all the atomic sentences that can be inferred in one step from the implication sentences and the atomic sentences already in KB. The function STANDARDIZE-VARIABLES replaces all variables in its arguments with new ones that have not been used before.

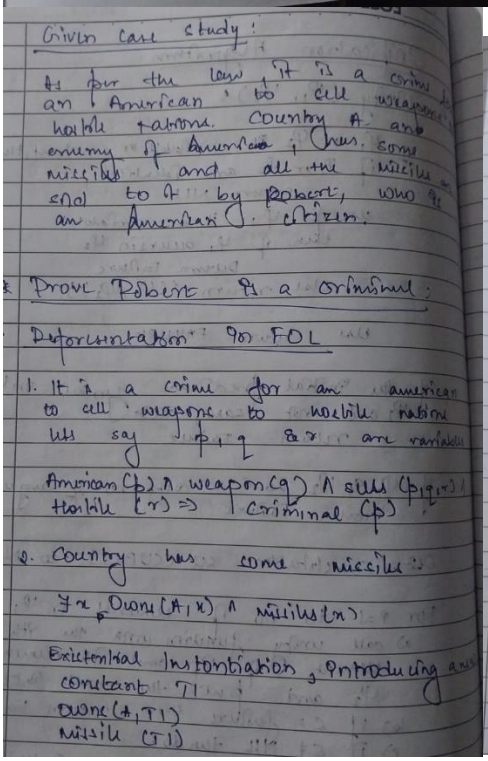
Algorithm:

function FOL-FC-ASK(KB, α) returns a substitution or false

inputs: KB, the knowledge base, a set of first order definite clauses
α, the query, an atomic sentence

local variables: new, the new sentences inferred on each iteration

repeat until new is empty
new ← { }
for each rule in KB do
(p1 ∧ ... ∧ pn ⇒ q) ← STANDARDIZE-VARIABLES(rule)
for each θ such that SUBST(θ, p1 ∧ ... ∧ pn) = SUBST(θ, p'1 ∧ ... ∧ p'n)
for some p'1, ..., p'n in KB
q' ← SUBST(θ, q)
if q' does not unify with some sentence already in KB or new then
add q' to new
φ ← UNIFY(q', α)
if φ is not fail then return φ
add new to KB
return false


Given Case study:

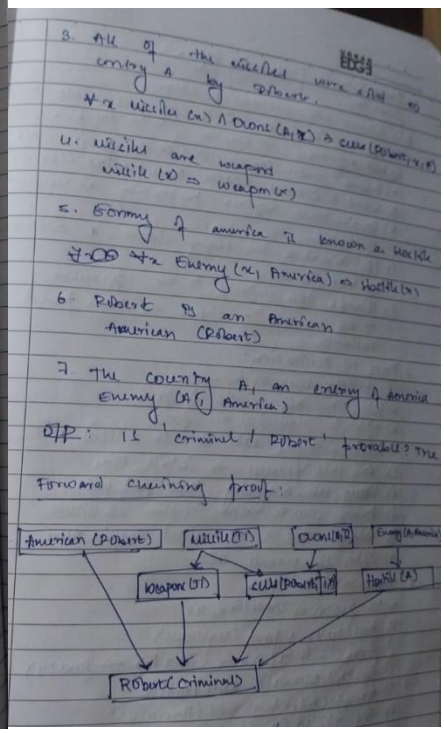
As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen.

Prove Robert is a criminal.

Representation in FOL

1. It is a crime for an american to sell weapons to hostile nations
Let us say p, q & r are variables
American(p) ∧ weapon(q) ∧ sells(p, q, r) ∧ hostile(r) ⇒ Criminal(p)

2. Country has some missiles.
∃x Owns(A, x) ∧ missiles(x)
Existential Instantiation, Introducing a new constant T1
Owns(A, T1)
missile(T1)


3. All of the missiles were sold to country A by Robert.
∀x missiles(x) ∧ Owns(A, x) ⇒ sells(Robert, x, A)

4. missiles are weapons
missile(x) ⇒ weapon(x)

5. Enemy of america is known as hostile
∀x Enemy(x, America) ⇒ Hostile(x)

6. Robert is an American
American(Robert)

7. The country A is an enemy of America
Enemy(A, America)

O/P: It is a criminal to Robert is probable? True

Forward chaining proof:

Code:

```
class KnowledgeBaseSystem:
    def __init__(self):
        self.known_facts = set()
        self.rules_list = []

    def insert_fact(self, fact):
        self.known_facts.add(fact)

    def insert_rule(self, rule):
        self.rules_list.append(rule)

    def deduce(self):
        new_inferences = True
        while new_inferences:
            new_inferences = False
            for rule in self.rules_list:
                if rule.check_and_apply(self.known_facts):
                    new_inferences = True

knowledge_base = KnowledgeBaseSystem()
knowledge_base.insert_fact("American(Robert)")
knowledge_base.insert_fact("Missile(T1)")
knowledge_base.insert_fact("Owns(A, T1)")
knowledge_base.insert_fact("Enemy(A, America)")


class InferenceRule:
    def __init__(self, conditions, result):
        self.conditions = conditions  # List of conditions
        self.result = result  # The result to derive if conditions are met

    def check_and_apply(self, facts):
        if all(condition in facts for condition in self.conditions):
            if self.result not in facts:
                facts.add(self.result)
                print(f"Derived: {self.result}")
                return True
        return False



knowledge_base.insert_rule(InferenceRule(["Missile(T1)"],"Weapon(T1)"))
knowledge_base.insert_rule(InferenceRule(["Enemy(A, America)"],"Hostile(A)"))
```

```
knowledge_base.insert_rule(InferenceRule(["Missile(T1)", "Owns(A, T1)"],"Sells(Robert,
T1, A)"))
knowledge_base.insert_rule(InferenceRule(
    ["American(Robert)", "Weapon(T1)", "Sells(Robert, T1, A)",
"Hostile(A)"],"Criminal(Robert)"))
knowledge_base.deduce()

if "Criminal(Robert)" in knowledge_base.known_facts:
    print("Conclusion: Robert is a criminal.")
else:
    print("Conclusion: Unable to prove Robert is a criminal.")
```

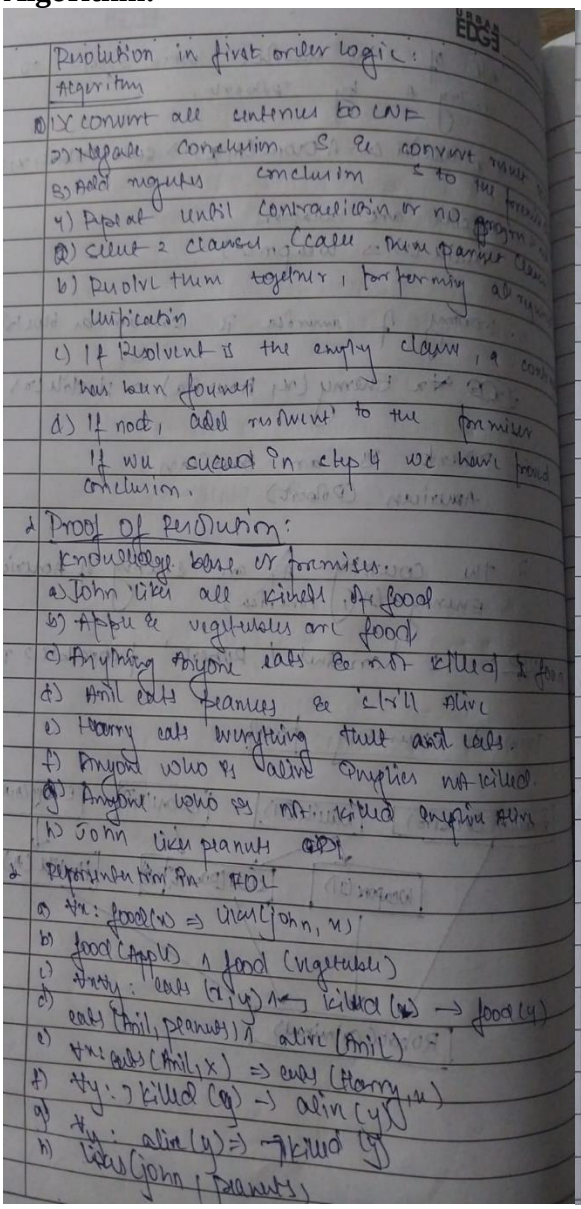
Output Snapshot:

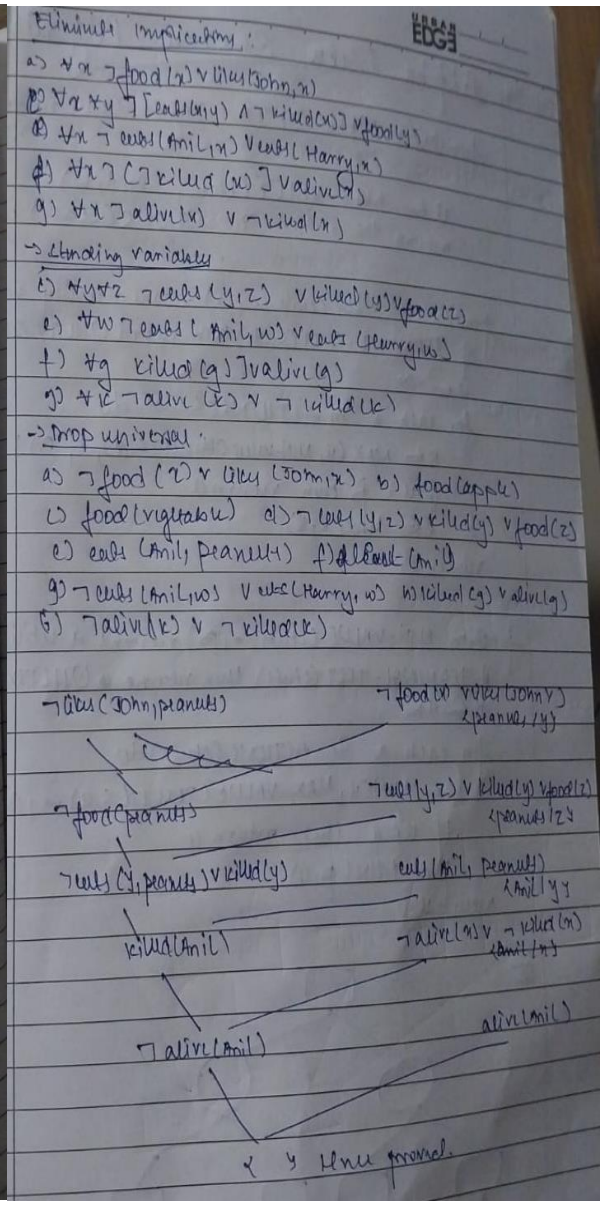
```
Derived: Weapon(T1)
Derived: Hostile(A)
Derived: Sells(Robert, T1, A)
Derived: Criminal(Robert)
Conclusion: Robert is a criminal.
```

Program9:

Create a knowledge base consisting of first order logic statements and prove the given query using Resolution.

Algorithm:





Code:

```
from itertools import combinations

def unify_sentences_var(var, x, theta):
    if var in theta:
        return unify_sentences(theta[var], x, theta)
    elif x in theta:
        return unify_sentences(var, theta[x], theta)
```

```
    else:
        theta[var] = x
        return theta

def resolve(sentence1, sentence2):
    resolvents = []
    for predicate1 in sentence1:
        for predicate2 in sentence2:
            theta = unify_sentences(predicate1, negate(predicate2))
            if theta is not None:
                new_sentence = (substitute(sentence1, theta) | substitute(sentence2, theta)) -
{predicate1, predicate2}
                resolvents.append(frozenset(new_sentence))
    return resolvents

def unify_sentences(x, y, theta={}):
    if theta is None:
        return None
    elif x == y:
        return theta
    elif isinstance(x, str) and x.islower():
        return unify_sentences_var(x, y, theta)
    elif isinstance(y, str) and y.islower():
        return unify_sentences_var(y, x, theta)
    elif isinstance(x, tuple) and isinstance(y, tuple) and len(x) == len(y):
        return unify_sentences(x[1:], y[1:], unify_sentences(x[0], y[0], theta))
    else:
        return None

def negate(predicate):
    return ('not', predicate) if isinstance(predicate, str) else predicate[1]

def substitute(sentence, theta):
    return {substitute_predicate(p, theta) for p in sentence}

def substitute_predicate(predicate, theta):
    if isinstance(predicate, str):
        return theta.get(predicate, predicate)
    else:
        return (predicate[0],) + tuple(theta.get(arg, arg) for arg in predicate[1:])

def proof_by_resolution(Knowledge_Base, query):
    negated_query = frozenset({negate(query)})
    sentences = Knowledge_Base | {negated_query}
    new_sentences = set()

    while True:
```

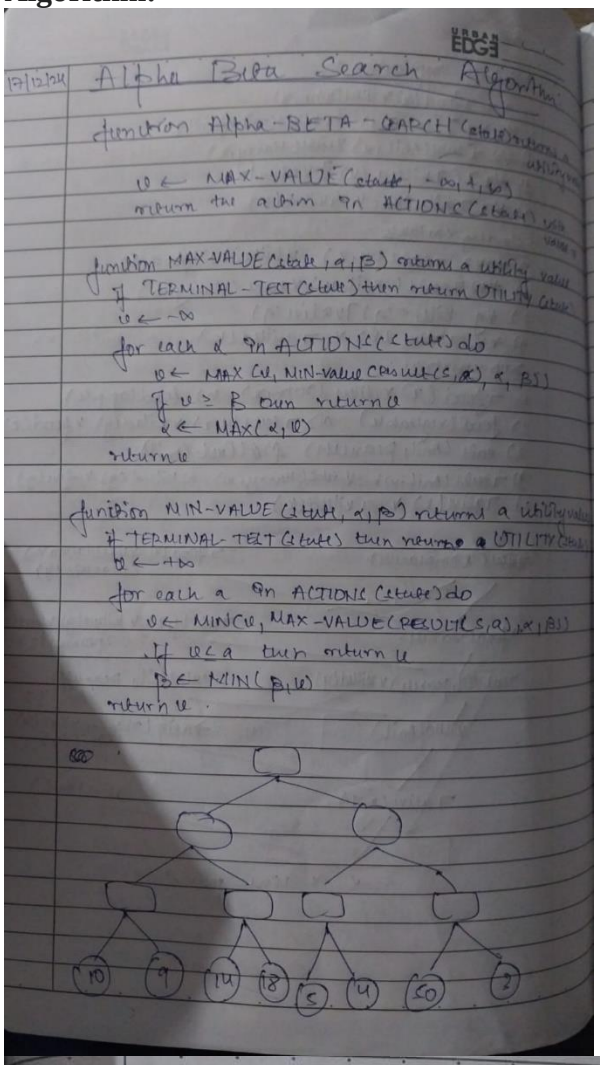
```
        for sentence1, sentence2 in combinations(sentences, 2):
            resolvents = resolve(sentence1, sentence2)
            if frozenset() in resolvents:
                return True
            new_sentences.update(resolvents)
        if new_sentences.issubset(sentences):
            return False
        sentences |= new_sentences

Knowledge_Base = {
    frozenset({('Mother', 'Leela', 'Oshin')}),
    frozenset({('Alive', 'Leela')}),
    frozenset({('not','Mother', 'x','y')}),
    frozenset({('Parent','x','y')}),
    frozenset({('not','Parent', 'w', 'z')}),
    frozenset({('not','Alive','w','z')}),
    frozenset({('Older','w','z')}),
}

query = ('Older', 'Leela', 'Older')
result = proof_by_resolution(Knowledge_Base, query)
if result:
    print("Leela is older than Oshin.\nProved by resolution.")
else:
    print("Cannot prove. Leela is not older than Oshin.")
```

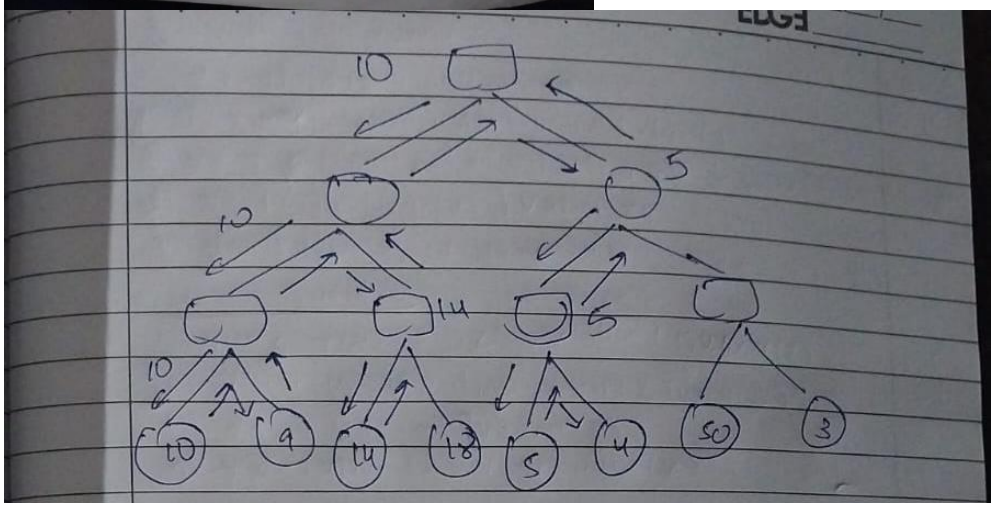
Output Snapshot:

```
Leela is older than Oshin.
Proved by resolution.
```

Program10:
Implement Alpha-Beta Pruning.

Algorithm:





Code:

```
class Node:
    def __init__(self, value=None, children=None):
        self.value = value
        self.children = children if children else []

def alpha_beta_pruning(node, depth, alpha, beta, maximizing_player):
    if not node.children or depth == 0:
        return node.value

    if maximizing_player:
        max_eval = float('-inf')
        for child in node.children:
            eval = alpha_beta_pruning(child, depth - 1, alpha, beta, False)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                print(f"Pruned at MAX node with alpha={alpha}, beta={beta}")
                break
        node.value = max_eval
        return max_eval
    else:
        min_eval = float('inf')
        for child in node.children:
            eval = alpha_beta_pruning(child, depth - 1, alpha, beta, True)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                print(f"Pruned at MIN node with alpha={alpha}, beta={beta}")
                break
        node.value = min_eval
        return min_eval


def print_tree(node, level=0):
    print("  " * level *2 + f"Value of Node: {node.value}")
    for child in node.children:
        print_tree(child, level + 1)


if __name__ == "__main__":
    tree = Node(None, [
        Node(None, [
            Node(None,[Node(10),Node(9)]),
            Node(None, [Node(14),Node(18)])
        ]),
        Node(None, [
```

```
            Node(None, [Node(5),Node(4)]),
            Node(None, [Node(50),Node(3)])
        ])
    ])

    print("Game Tree Before Alpha-Beta Pruning:")
    print_tree(tree)
    final_value = alpha_beta_pruning(tree, depth=3, alpha=float('-inf'), beta=float('inf'),
maximizing_player=True)
    print("\nGame Tree After Alpha-Beta Pruning:")
    print_tree(tree)
    print("\nFinal Value at MAX node:", final_value)
```

Output Snapshot:

```
Game Tree Before Alpha-Beta Pruning:
Value of Node: None
    Value of Node: None
        Value of Node: None
            Value of Node: 10
            Value of Node: 9
        Value of Node: None
            Value of Node: 14
            Value of Node: 18
    Value of Node: None
        Value of Node: None
            Value of Node: 5
            Value of Node: 4
        Value of Node: None
            Value of Node: 50
            Value of Node: 3
Pruned at MAX node with alpha=14, beta=10
Pruned at MIN node with alpha=10, beta=5

Game Tree After Alpha-Beta Pruning:
Value of Node: 10
    Value of Node: 10
        Value of Node: 10
            Value of Node: 10
            Value of Node: 9
        Value of Node: 14
            Value of Node: 14
            Value of Node: 18
    Value of Node: 5
        Value of Node: 5
            Value of Node: 5
            Value of Node: 4
        Value of Node: None
            Value of Node: 50
            Value of Node: 3

Final Value at MAX node: 10
```